gemstone-rs Shared Core Integration

How gemstone-py-native can wrap the Rust core

Generated from the Markdown sources in gemstone-rs/docs.

Contents

Shared Core Integration

The long-term direction is:

```
gemstone-gci
  Low-level dynamic libgcirpc loader and raw GCI calls.

gemstone-rs
  Safe Rust API: Config, Session, Oop, Value, browser, codegen, BridgeRoot.
  Also exposes gemstone_rs::py_native as a narrow adapter contract.

gemstone-py-native
  Thin PyO3 wrapper around the Rust core.

gemstone-py
  Python API, async/web adapters, examples, docs, and compatibility layer.
```

This keeps Python and Rust from growing separate native bridges.

What Should Move Into Rust

- dynamic GCI library loading
- OOP constants and conversions
- session login/logout
- eval, execute, perform
- global get/put
- string, symbol, array, and dictionary marshalling
- transaction commit/abort
- browser primitives
- codegen discovery primitives

What Should Stay Python-Specific

- Pythonic `Session` and async wrappers
- FastAPI, Litestar, Django examples
- Python package extras such as `gemstone-py[fast]`
- backward-compatible return behavior
- Python docs and notebooks

PyO3 Wrapper Shape

`gemstone-py-native` should become a small adapter:

```
Python call -> PyO3 wrapper -> gemstone-rs Session -> gemstone-gci -> libgcirpc
```

The Rust side now exposes `gemstone_rs::py_native`, which is deliberately plain Rust and dependency-free. A PyO3 crate can wrap these types without depending on internal `Session` details:

- `PyNativeConfig`
- `PyNativeConfigSummary`
- `PyNativeValue`
- `PyNativeErrorInfo`
- `PyNativeSession`
- `capabilities()`

Print the adapter contract from the CLI when a wrapper build, CI job, or documentation generator needs a stable machine-readable surface:

```
gemstone-rs py-native capabilities
gemstone-rs py-native capabilities --json
```

Run the dry-run contract check from a source checkout when you also want to exercise the example binary:

```
cargo run -p gemstone-rs --example python_native_adapter -- --dry-run
```

Run it against a live stone:

```
export GS_LIB=/opt/gemstone/product/lib
export GS_STONE=gs64stone
export GS_USERNAME=DataCurator
export GS_PASSWORD=swordfish
cargo run -p gemstone-rs --example python_native_adapter
```

The wrapper should expose stable operations first:

- `login(config)`
- `eval(source)`
- `execute(source)`
- `value_to_oop(value)`
- `perform(oop, selector, args)`
- `commit()`

- `abort()`
- `logout()`

Only after that should it expose higher-level browser, codegen, and BridgeRoot operations.

PyO3 Sketch

The future `gemstone-py-native` crate should be thin:

```
use gemstone_rs::py_native::{PyNativeConfig, PyNativeSession};

struct NativeSession {
    inner: PyNativeSession,
}

impl NativeSession {
    fn login(config: PyNativeConfig) -> gemstone_rs::Result<Self> {
        Ok(Self {
            inner: PyNativeSession::login(config)?,
        })
    }

    fn eval(&mut self, source: &str) -> gemstone_rs::Result<String> {
        Ok(format!("{:?}", self.inner.eval(source)?))
    }
}
```

Actual PyO3 code should translate `PyNativeValue` into Python objects and translate `PyNativeErrorInfo` into Python exceptions. Keep `PyNativeSession` unsendable unless a dedicated worker-thread wrapper is used.

Migration Plan

1. Keep `gemstone-rs` independent and publishable.
2. Wrap `gemstone_rs::py_native` from the existing `gemstone-py-native` PyO3

   crate.

3. Replace duplicated native loading code in `gemstone-py-native`.
4. Run the existing `gemstone-py` native backend and live tests through the

   Rust-backed native path.

5. Keep pure Python fallback behavior and current sync return behavior

   backward compatible.

The main design rule: Rust owns the native bridge; Python owns Python ergonomics.

This PDF was generated locally from `docs/`. If you edit the Markdown sources, rerun `python docs/build_pdf_docs.py`.